

MODULE 3 — MODERN FILE HANDLING USING `java.nio.file` PACKAGE

Learning Objectives

At the end of this module, students should be able to:

1. Understand the `java.nio.file` package and why it was introduced.
2. Work with `Path`, `Paths`, and `Files` classes.
3. Perform operations like file creation, copying, moving, deleting, and checking file existence.
4. Read and write files using modern NIO methods.
5. Handle file metadata such as size, attributes, and permissions.

3.1 Introduction to NIO (New I/O)

What is NIO?

NIO stands for **New Input/Output**, introduced in **Java 7** under the package `java.nio.file`.

It provides a **more flexible, faster, and powerful** way to handle files and directories than the traditional `java.io` classes.

Why NIO?

Feature	<code>java.io</code>	<code>java.nio.file</code>
File Representation	<code>File</code> object	<code>Path</code> interface
Performance	Slower	Faster (buffered and channel-based)
File Operations	Limited	Rich and easy-to-use utilities

Feature	java.io	java.nio.file
Exception Handling	Manual	Streamlined
Symbolic Links	No support	Supported
Atomic Operations	No	Yes

3.2 Understanding Path and Paths

◆ Path

- Represents the **location** of a file or directory.
- Works across operating systems.

◆ Paths

- A utility class used to **create Path objects**.



Example 1: Creating a Path

```
import java.nio.file.Path;
import java.nio.file.Paths;

public class PathExample {
    public static void main(String[] args) {
        Path path = Paths.get("C:/Users/Public/test.txt");
        System.out.println("File Name: " + path.getFileName());
        System.out.println("Parent: " + path.getParent());
        System.out.println("Absolute Path: " + path.toAbsolutePath());
        System.out.println("Root: " + path.getRoot());
    }
}
```

Explanation:

- `Paths.get()` creates a Path instance.
- You can extract details like file name, parent directory, root, etc.

**Tip:**

You can also use relative paths:

```
Path relativePath = Paths.get("data/info.txt");
```



3.3 The Files Class

The `Files` class contains **static utility methods** for file manipulation such as creating, copying, moving, reading, and deleting.



Example 2: Creating a New File

```
import java.nio.file.*;

public class CreateFileExample {
    public static void main(String[] args) {
        Path filePath = Paths.get("newfile.txt");
        try {
            Files.createFile(filePath);
            System.out.println("File created successfully.");
        } catch (FileAlreadyExistsException e) {
            System.out.println("File already exists!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Explanation:

- `Files.createFile()` creates a new file.
- Throws `FileAlreadyExistsException` if it already exists.



Example 3: Creating a Directory

```
import java.nio.file.*;

public class CreateDirectoryExample {
    public static void main(String[] args) {
        Path dirPath = Paths.get("Documents/JavaFiles");
        try {
            Files.createDirectories(dirPath);
            System.out.println("Directories created successfully!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Note:

- `createDirectory()` → creates one directory.
- `createDirectories()` → creates parent directories if they don't exist.



3.4 Checking File Properties

Use methods from `Files` to check file details:

Example 4: Checking File Information

```
import java.nio.file.*;

public class FileInfoExample {
    public static void main(String[] args) {
        Path file = Paths.get("newfile.txt");

        System.out.println("Exists: " + Files.exists(file));
        System.out.println("Readable: " + Files.isReadable(file));
        System.out.println("Writable: " + Files.isWritable(file));
        System.out.println("Executable: " + Files.isExecutable(file));
        System.out.println("Is Directory: " + Files.isDirectory(file));
    }
}
```



3.5 Copying, Moving, and Deleting Files



Example 5: Copying Files

```
import java.nio.file.*;

public class CopyFileExample {
    public static void main(String[] args) {
        Path source = Paths.get("data.txt");
        Path destination = Paths.get("backup/data_copy.txt");

        try {
            Files.copy(source, destination, StandardCopyOption.REPLACE_EXISTING);
            System.out.println("File copied successfully.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



Example 6: Moving Files

```
import java.nio.file.*;

public class MoveFileExample {
    public static void main(String[] args) {
        Path source = Paths.get("data.txt");
        Path target = Paths.get("archive/data.txt");

        try {
            Files.move(source, target, StandardCopyOption.REPLACE_EXISTING);
            System.out.println("File moved successfully.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



Example 7: Deleting Files

```
import java.nio.file.*;

public class DeleteFileExample {
    public static void main(String[] args) {
        Path file = Paths.get("oldfile.txt");
        try {
            Files.deleteIfExists(file);
            System.out.println("File deleted successfully.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



3.6 Reading and Writing Files with NIO

Instead of using `FileReader` and `FileWriter`, you can use the `Files` class for quick operations.



Example 8: Writing to a File

```
import java.nio.file.*;
import java.io.IOException;
import java.util.*;

public class NIOWriteExample {
    public static void main(String[] args) {
        Path file = Paths.get("notes.txt");
        List<String> lines = Arrays.asList("Hello, Java NIO!", "It's fast and efficient.");
        try {
            Files.write(file, lines);
            System.out.println("File written successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



Example 9: Reading a File

```
import java.nio.file.*;
import java.io.IOException;
import java.util.List;

public class NIOReadExample {
    public static void main(String[] args) {
        Path file = Paths.get("notes.txt");
        try {
            List<String> lines = Files.readAllLines(file);
            for (String line : lines) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



Example 10: Reading Large Files Using Stream

```
import java.nio.file.*;
import java.util.stream.Stream;

public class StreamReadExample {
    public static void main(String[] args) {
        Path file = Paths.get("notes.txt");
        try (Stream<String> lines = Files.lines(file)) {
            lines.forEach(System.out::println);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Explanation:

- `Files.lines()` returns a Stream of file lines.
- Efficient for large files as it reads lazily (line by line).



3.7 Getting File Metadata

Use `Files.getAttribute()` or helper methods to get file size, creation time, etc.

Example 11: File Metadata

```
import java.nio.file.*;
import java.nio.file.attribute.*;
import java.io.IOException;

public class FileMetadataExample {
    public static void main(String[] args) {
        Path path = Paths.get("notes.txt");
        try {
            BasicFileAttributes attr = Files.readAttributes(path, BasicFileAttributes.class);
            System.out.println("Creation Time: " + attr.creationTime());
            System.out.println("Last Modified: " + attr.lastModifiedTime());
            System.out.println("Size: " + attr.size() + " bytes");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



3.8 Classwork / Exercises

1. Create a directory named `Assignments` .
2. Inside it, create a text file named `task.txt` .
3. Write your name and course title inside the file.
4. Copy the file to a folder named `Backup` .
5. Move it from `Backup` to `Archive` .
6. Display the file size and creation date.
7. Finally, delete the `Archive` folder programmatically.

3.9 Challenge Task

Create a program that:

- Lists all `.txt` files inside a given directory.
- Displays their size and last modified time.
- Saves this report to a file named `report.txt`.

Hint: Use

```
Files.newDirectoryStream(Path dir, "*.txt");
```

3.10 Summary of Module 3

Class	Description	Example
Path	Represents a file or directory location	<code>Paths.get("notes.txt")</code>
Paths	Creates Path objects	<code>Paths.get()</code>
Files	Contains static methods for file operations	<code>Files.copy()</code> , <code>Files.move()</code>
BasicFileAttributes	Reads metadata about a file	<code>Files.readAttributes()</code>
StandardCopyOption	Defines copy/move options	<code>REPLACE_EXISTING</code>

End of Module 3 Summary

In this module, you learned:

- The **modern way of handling files** using the `java.nio.file` package.
- How to create, copy, move, read, and delete files with ease.
- How to read large files and access file metadata.

Next, in **Module 4**, we'll explore **Object Serialization and Deserialization in Java** — how to write and read **objects** to and from files.